Programming Assignment: Tomasulo Algorithm

Multi-issue (1–4 instructions per cycle, in order)

Ilan Lipsky

GitHub repository:
`https://github.com/CoinismRS/LipskyIlan_COSC6385_Programming_Tomasulo_HW`

1 Introduction

Tomasulo's algorithm schedules instructions dynamically: instructions wait in reservation stations until their operands are broadcast on the common data bus (CDB), execute out of order, and commit in order through the reorder buffer (ROB). The provided Python simulator implements this for *one* instruction per cycle. In this assignment I modified that code to:

1. **issue 1–4 instructions per cycle** in program order (`Issue width` in the input file, default 1), **commit** up to the same number per cycle, and broadcast on a configurable number of **CDBs** (default 1);
2. support **several functional units** per type (the "# of FUs" column), with pipelined FP units and unpipelined integer/address adders;
3. add the **branch unit** described in the handout, which the provided code did not have (it stopped fetching at every `Bne`): an 8-entry BTB with a 1-bit predictor, speculative issue past branches, and misprediction recovery with fetch restarting at cycle $n+2$;
4. read the **whole configuration from the input file** (it was hard-coded in `main.py`) and print the final **register and memory** values;
5. fix several bugs in the original code that showed up once more than one instruction was in flight (Section 2.8).

I kept the original program structure: the same files, the same stage functions, the same record types (`rs_entry`, `ld_sd_entry`, `ROB_entry`, ...), the same `ROB`n / `ldsd`n tags, the RAT that holds either a tag or a value, the results buffer feeding the CDB, and the original timing conventions. The only new file is `branch.py`. About 75% of the original source lines are unchanged (382 of 511 distinct lines). The provided code is included in `original_code/` for comparison.

What works. Issue widths 1–4 with in-order issue and structural-hazard stalls; commit width and CDB count; multiple FUs; `Beq`/`Bne` with BTB prediction, speculation and recovery; store-to-load forwarding; R0 hard-wired to 0. Every test bench at every width ends with the same registers and memory as a sequential reference interpreter, and so do 20,000 randomly generated programs (Section 4).

Limitations. Loads never pass a store with an unknown address (conservative, as in the original). A store is sent to memory only when it is the oldest instruction in the ROB. Memory has one value per byte address (as in the original), so word overlap is not modelled.

2 Code changes

The excerpts below are copied from the source files by the report's build script.

2.1 Issuing multiple instructions (`issue.py`)

The original `issue()` fetched one instruction. I renamed its body to `issue_one()` (which now reports whether the instruction could be issued) and wrote a new `issue()` that calls it up to `issue_width` times. Issue stops at the first instruction that cannot get a ROB entry and a reservation-station / load-store-queue entry, so instructions never issue out of order. A branch predicted taken also ends the group, and its target is fetched next cycle. Because each call reads the RAT *before* renaming its destination, dependences between instructions issued in the same cycle are handled by the original renaming code unchanged.

Initial version

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    op = ins.split(' ')[0]
    '''decode'''
    # LD/SD instructions
    if (op=='Ld')|(op=='Sd'):
        # check space
        if (len(ROB)<size_ROB)&(len(ld_sd_queue)<size_ld_sd_queue):
            # find ldsd_tag
            ldsd_tag = find_tag_new_ldsd_entry(ld_sd_queue, size_ld_sd_queue)
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, ldsd_tag)
            # put ins into ld_sd_queue
            put_ins_into_ldsd(ldsd_tag, ld_sd_queue, ins, ROB, rat_int, rat_fp)
            # update RAT
            update_rat(ROB, rat_int, rat_fp)
            # PC
            PC.PC +=1
    # integer adder instructions
    elif (op=='Add')|(op=='Addi')|(op=='Sub'):
        # check space
        if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
            # put ins into rs_int_adder
            index = check_rs_space(rs_int_adder)
            put_ins_into_rs(rs_int_adder, index, ins, ROB, rat_int, rat_fp)
            # update RAT
            update_rat(ROB, rat_int, rat_fp)
            # PC
            PC.PC +=1
    # fp adder instructions
    elif (op=='Add.d')|(op=='Sub.d'):
        # check space
        if (len(ROB)<size_ROB)&(check_rs_space(rs_fp_adder)>=0):
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
```

```
            # put ins into rs_fp_adder
            index = check_rs_space(rs_fp_adder)
            put_ins_into_rs(rs_fp_adder, index, ins, ROB, rat_int, rat_fp)
            # update RAT
            update_rat(ROB, rat_int, rat_fp)
            # PC
            PC.PC +=1
    # fp multiplier instructions
    elif (op=='Mult.d'):
        # check space
        if (len(ROB)<size_ROB)&(check_rs_space(rs_fp_multi)>=0):
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
            # put ins into rs_fp_multi
            index = check_rs_space(rs_fp_multi)
            put_ins_into_rs(rs_fp_multi, index, ins, ROB, rat_int, rat_fp)
            # update RAT
            update_rat(ROB, rat_int, rat_fp)
            # PC
            PC.PC +=1
    # Bne
    elif (op=='Bne'):
        # check space
        if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
            # put ins into rs_int_adder
            index = check_rs_space(rs_int_adder)
            put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
            # PC
            PC.valid = 0
    else:
        pass
```

My version

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp, issue_width, btb, stats):
    if cycle < PC.resume:
        return
    count = 0
    while (count < issue_width) & (0 <= PC.PC < len(instructions)):
        old_PC = PC.PC
        issued = issue_one(cycle, PC, instructions, ROB, size_ROB,
                           rs_int_adder, rs_fp_adder, rs_fp_multi,
                           ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp, btb)
        if not issued:
            break                       # structural hazard: in-order issue stalls
        count += 1
        stats['issued'] += 1
        if PC.PC != old_PC + 1:
            break                       # predicted-taken branch ends the group
    stats['issue_per_cycle'][cycle] = count
```

In `issue_one()` the `Bne` case, which used to stop fetch with `PC.valid = 0`, now predicts the next PC and keeps going (and handles `Beq`):

```
            # PC: originally fetch stopped here (PC.valid = 0) until the branch
            # resolved; now the BTB predicts the next PC and issue continues
            ROB[-1].pred_PC = predict(btb, PC.PC)
            PC.PC = ROB[-1].pred_PC
            issued = True
```

2.2 Several CDBs (`wb.py`)

The single CDB latch became a list of latches. Each broadcasts its result, then each empty one takes the next result from the results buffer (still FIFO, as in the original).
Initial version

```
def wb(cdb, rat_int, rat_fp,
        rs_int_adder, rs_fp_adder, rs_fp_multi,
        ld_sd_queue, ROB, cycle,
        results_buffer):
    # broadcast
    if cdb.valid == 1:
        boroadcast(cdb, rat_int, rat_fp,
                    rs_int_adder, rs_fp_adder, rs_fp_multi,
                    ld_sd_queue, ROB, cycle)
        cdb.valid = 0
    # fetch data from results queue
    if cdb.valid == 0:
        if len(results_buffer)>0:
            cdb.valid = 1
            cdb.value = results_buffer[0].value
            cdb.dest_tag = results_buffer[0].dest_tag
            results_buffer.popleft()
```

My version

```
def wb(cdb, rat_int, rat_fp,
        rs_int_adder, rs_fp_adder, rs_fp_multi,
        ld_sd_queue, ROB, cycle,
        results_buffer):
    for bus in cdb:
        # broadcast
        if bus.valid == 1:
            boroadcast(bus, rat_int, rat_fp,
                        rs_int_adder, rs_fp_adder, rs_fp_multi,
                        ld_sd_queue, ROB, cycle)
            bus.valid = 0
    for bus in cdb:
        # fetch data from results queue
        if bus.valid == 0:
            if len(results_buffer)>0:
                bus.valid = 1
                bus.value = results_buffer[0].value
                bus.dest_tag = results_buffer[0].dest_tag
                results_buffer.popleft()
```

In `boroadcast()` the RAT is now only updated if it still maps the register to the broadcasting ROB entry (bug fix 1 in Section 2.8).
Initial version

```
def boroadcast(cdb, rat_int, rat_fp,
                rs_int_adder, rs_fp_adder, rs_fp_multi,
                ld_sd_queue, ROB, cycle):
    # dest_tag
```

```
    dest_tag = cdb.dest_tag
    index = find_ROB_entry(ROB, dest_tag)
    reg_tag = ROB[index].dest_tag
    # rat_int
    if reg_tag[0] == 'R':
        rat_int[int(reg_tag[1:])] = cdb.value
    # rat_fp
    if reg_tag[0] == 'F':
        rat_fp[int(reg_tag[1:])] = cdb.value
    # rs list
    rs_list = [rs_int_adder, rs_fp_adder, rs_fp_multi]
    for rs in rs_list:
        for element in rs:
            if (element.tag_1st==dest_tag)&(element.valid_1st==0):
                element.value_1st = cdb.value
                element.valid_1st = 1
            if (element.tag_2nd==dest_tag)&(element.valid_2nd==0):
                element.value_2nd = cdb.value
                element.valid_2nd = 1
    # ld_sd_queue
    for element in ld_sd_queue:
        if (element.data==dest_tag)&(element.op=='Sd'):
            element.data = cdb.value
        if (element.reg_tag==dest_tag)&(element.valid==0):
            element.reg_value = cdb.value
            element.valid = 1
    # ROB
    for element in ROB:
        if element.ROB_tag == dest_tag:
            element.value = cdb.value
            element.cdb.append(cycle)
```

My version

```
def boroadcast(cdb, rat_int, rat_fp,
               rs_int_adder, rs_fp_adder, rs_fp_multi,
               ld_sd_queue, ROB, cycle):
    # dest_tag
    dest_tag = cdb.dest_tag
    index = find_ROB_entry(ROB, dest_tag)
    reg_tag = ROB[index].dest_tag
    # rat_int
    if reg_tag[0] == 'R':
        if rat_int[int(reg_tag[1:])] == dest_tag:
            rat_int[int(reg_tag[1:])] = cdb.value
    # rat_fp
    if reg_tag[0] == 'F':
        if rat_fp[int(reg_tag[1:])] == dest_tag:
            rat_fp[int(reg_tag[1:])] = cdb.value
    # rs list
    rs_list = [rs_int_adder, rs_fp_adder, rs_fp_multi]
    for rs in rs_list:
        for element in rs:
            if (element.tag_1st==dest_tag)&(element.valid_1st==0):
                element.value_1st = cdb.value
                element.valid_1st = 1
            if (element.tag_2nd==dest_tag)&(element.valid_2nd==0):
                element.value_2nd = cdb.value
                element.valid_2nd = 1
    # ld_sd_queue
    for element in ld_sd_queue:
```

```
        if (element.data==dest_tag)&(element.op=='Sd'):
            element.data = cdb.value
        if (element.reg_tag==dest_tag)&(element.valid==0):
            element.reg_value = cdb.value
            element.valid = 1
    # ROB
    for element in ROB:
        if element.ROB_tag == dest_tag:
            element.value = cdb.value
            element.cdb.append(cycle)
```

2.3 Committing multiple instructions (`commit.py`)

Up to `commit_width` instructions leave the ROB head per cycle. I kept the original convention that an instruction broadcast in cycle n commits in cycle $n+1$. Branches now commit only once resolved and get a commit cycle. A store commits when `mem.py` sends it to memory, which happens only when it is the ROB head.

Initial version

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):
    if len(ROB)>0:
        if instructions[ROB[0].PC].split(' ')[0]=='Bne':
            if len(ROB[0].exe)!=0:
                entry = ROB.popleft()
                print_ROB(entry, instructions)
        if (len(ROB)>0)&(instructions[ROB[0].PC].split(' ')[0]!='Bne'):
            if len(ROB[0].cdb)!=0: # broadcasted instructions
                ROB[0].commit.append(cycle+1)
                entry = ROB.popleft()
                modify_arch_reg(entry, reg_int, reg_fp)
                print_ROB(entry, instructions)
            elif len(ROB[0].commit)!=0: # Sd
                entry = ROB.popleft()
                print_ROB(entry, instructions)
                if len(ROB)>0:
                    if len(ROB[0].cdb)!=0: # broadcasted instructions
                        ROB[0].commit.append(cycle+1)
                        entry = ROB.popleft()
                        modify_arch_reg(entry, reg_int, reg_fp)
                        print_ROB(entry, instructions)
            else:
                pass
```

My version

```
def commit(ROB, reg_int, reg_fp, cycle, instructions, commit_width, committed):
    count = 0
    while (count < commit_width) & (len(ROB) > 0):
        op = instructions[ROB[0].PC].split(' ')[0]
        if (op=='Bne')|(op=='Beq'):
            ready = len(ROB[0].exe) == 2           # resolved
        elif op == 'Sd':
            ready = ROB[0].sd_sent                  # sent to memory by mem.py
        else:
            ready = len(ROB[0].cdb) != 0            # broadcasted instructions
        if not ready:
            break
        entry = ROB.popleft()
        entry.commit.append(cycle+1)
```

```
        if (op!='Bne')&(op!='Beq')&(op!='Sd'):
            if entry.fault is not None:
                raise ValueError('%s: load from invalid address %s'
                                 % (instructions[entry.PC], entry.fault))
            modify_arch_reg(entry, reg_int, reg_fp)
        committed.append(entry)
        count += 1
```

2.4 Multiple functional units and instruction selection (`exe.py`)

Each FU type is now a list of FIFO queues, one per unit. Every cycle, each free unit first fetches the *oldest* ready reservation-station entry (the original took the lowest RS index, whatever its age), and then every unit executes exactly once. The original executed a unit a second time when it fetched a "waiting" instruction (bug fix 5). The integer adder only accepts a new instruction when it is empty (unpipelined, per the handout); the FP units accept one per cycle (pipelined).

Initial version

```
def check_valid_ins_in_rs(rs):
    index = -1
    if len(rs)!=0:
        for i in range(len(rs)):
            if (rs[i].valid_1st==1)&(rs[i].valid_2nd==1)&(rs[i].busy==1):
                index = i
                break
    return index
```

My version

```
def check_valid_ins_in_rs(rs, ROB, cycle):
    index = -1
    oldest = len(ROB)
    if len(rs)!=0:
        for i in range(len(rs)):
            if (rs[i].valid_1st==1)&(rs[i].valid_2nd==1)&(rs[i].busy==1):
                position = find_ROB_entry(ROB, rs[i].dest_tag)
                if (ROB[position].issue[0] < cycle) & (position < oldest):
                    index = i
                    oldest = position
    return index
```

Initial version

```
def exe(fu_int_adder, time_fu_int_adder,
        fu_fp_adder, time_fu_fp_adder,
        fu_fp_multi, time_fu_fp_multi, results_buffer,
        rs_int_adder, rs_fp_adder, rs_fp_multi,
        ld_sd_exe, time_ld_sd_exe, ld_sd_queue,
        cycle, ROB, PC):
    '''execution in fu and ld_sd address calculation'''
    ld_sd_execution(ld_sd_exe, time_ld_sd_exe, ld_sd_queue, ROB, cycle)
    # functional units
    fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
    fu_exe(fu_fp_adder, results_buffer, ROB, time_fu_fp_adder, cycle, PC)
    fu_exe(fu_fp_multi, results_buffer, ROB, time_fu_fp_multi, cycle, PC)
    '''fetch instructions from rs and ld_sd_queue'''
    # from ld_sd_queue
    # check valid ins in ld_sd_queue
    index = check_valid_ins_in_ldsd(ld_sd_queue)
    # put ins into ld_sd_exe
    if (index>=0)&(ld_sd_exe.busy==0):
```

```
        ld_sd_exe.busy = 1
        ld_sd_exe.cycle = 0
        ld_sd_exe.value1 = ld_sd_queue[index].reg_value
        ld_sd_exe.value2 = ld_sd_queue[index].immediate
        ld_sd_exe.dest_tag = ld_sd_queue[index].ld_sd_tag
        if ROB[find_ROB_entry(ROB, ld_sd_queue[index].dest_tag)].issue[0] < cycle:
            ld_sd_execution(ld_sd_exe, time_ld_sd_exe, ld_sd_queue, ROB, cycle)
    # from rs
    # int_adder
    # fetch valid instruction
    if check_valid_ins_in_rs(rs_int_adder)>=0:
        index = check_valid_ins_in_rs(rs_int_adder)
        fu_int_adder.append(fu_entry())
        add_entry_into_fu(fu_int_adder, rs_int_adder[index])
        # check if it's a waiting instruction
        if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
            fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
        # remove ins from rs
        rs_int_adder[index].busy = 0
    # fp_adder
    if check_valid_ins_in_rs(rs_fp_adder)>=0:
        index = check_valid_ins_in_rs(rs_fp_adder)
        fu_fp_adder.append(fu_entry())
        add_entry_into_fu(fu_fp_adder, rs_fp_adder[index])
        if ROB[find_ROB_entry(ROB, rs_fp_adder[index].dest_tag)].issue[0] < cycle:
            fu_exe(fu_fp_adder, results_buffer, ROB, time_fu_fp_adder, cycle, PC)
        # remove ins from rs
        rs_fp_adder[index].busy = 0
    # fp_multi
    if check_valid_ins_in_rs(rs_fp_multi)>=0:
        index = check_valid_ins_in_rs(rs_fp_multi)
        fu_fp_multi.append(fu_entry())
        add_entry_into_fu(fu_fp_multi, rs_fp_multi[index])
        if ROB[find_ROB_entry(ROB, rs_fp_multi[index].dest_tag)].issue[0] < cycle:
            fu_exe(fu_fp_multi, results_buffer, ROB, time_fu_fp_multi, cycle, PC)
        # remove ins from rs
        rs_fp_multi[index].busy = 0
```

My version

```
def fetch_into_fu(fu, rs, ROB, cycle):
    index = check_valid_ins_in_rs(rs, ROB, cycle)
    if index >= 0:
        fu.append(fu_entry())
        add_entry_into_fu(fu, rs[index])
        # remove ins from rs
        rs[index].busy = 0

def exe(fu_int_adder, time_fu_int_adder,
        fu_fp_adder, time_fu_fp_adder,
        fu_fp_multi, time_fu_fp_multi, results_buffer,
        rs_int_adder, rs_fp_adder, rs_fp_multi,
        ld_sd_exe, time_ld_sd_exe, ld_sd_queue,
        cycle, ROB):
    branch_events = []
    '''fetch instructions from ld_sd_queue into free address adders, then execute'''
    for unit in ld_sd_exe:
        if unit.busy == 0:
            index = check_valid_ins_in_ldsd(ld_sd_queue, ROB, cycle)
            if index >= 0:
                unit.busy = 1
```

```
                unit.cycle = 0
                unit.value1 = ld_sd_queue[index].reg_value
                unit.value2 = ld_sd_queue[index].immediate
                unit.dest_tag = ld_sd_queue[index].ld_sd_tag
                ld_sd_queue[index].exe_busy = 1
        ld_sd_execution(unit, time_ld_sd_exe, ld_sd_queue, ROB, cycle)
    '''fetch instructions from rs, then execute every functional unit once'''
    # int adder: unpipelined -> only fetch when the unit is empty
    for fu in fu_int_adder:
        if len(fu) == 0:
            fetch_into_fu(fu, rs_int_adder, ROB, cycle)
        fu_exe(fu, results_buffer, ROB, time_fu_int_adder, cycle, branch_events)
    # fp adder / multiplier: pipelined -> one new instruction per cycle
    for fu in fu_fp_adder:
        fetch_into_fu(fu, rs_fp_adder, ROB, cycle)
        fu_exe(fu, results_buffer, ROB, time_fu_fp_adder, cycle, branch_events)
    for fu in fu_fp_multi:
        fetch_into_fu(fu, rs_fp_multi, ROB, cycle)
        fu_exe(fu, results_buffer, ROB, time_fu_fp_multi, cycle, branch_events)
    return branch_events
```

2.5 Branch prediction and recovery (new `branch.py`)

When a branch finishes EX, `fu_exe()` no longer moves the PC. Instead it reports the outcome. `main.py` then calls the new function `resolve_branches()`, which updates the BTB and checks the prediction:

Initial version

```
            elif (fu[0].op=='Bne'):
                fu_results.pop()
                if fu[0].value1 == fu[0].value2:
                    PC.PC += 1
                    PC.valid = 1
                else:
                    index = find_ROB_entry(ROB, fu[0].dest_tag)
                    offset = ROB[index].dest_tag
                    PC.PC = int(PC.PC + 1 + offset/4)
                    PC.valid = 1
```

My version

```
            elif (fu[0].op=='Bne')|(fu[0].op=='Beq'):
                # branch resolved: report (ROB tag, taken) to the branch unit,
                # which checks the prediction (branch.py)
                fu_results.pop()
                equal = (fu[0].value1 == fu[0].value2)
                taken = equal if fu[0].op=='Beq' else (not equal)
                branch_events.append((fu[0].dest_tag, taken))
```

New function

```
def predict(btb, pc):
    entry = btb[pc % len(btb)]
    if entry.valid == 1 and entry.tag == pc and entry.taken == 1:
        return entry.target
    return pc + 1
```

New function

```
def resolve_branches(events, hw, cycle):
    # oldest first; a branch squashed by an older one is ignored
```

```
    events = [e for e in events if find_ROB_index(hw.ROB, e[0]) >= 0]
    events.sort(key=lambda e: find_ROB_index(hw.ROB, e[0]))
    for rob_tag, taken in events:
        index = find_ROB_index(hw.ROB, rob_tag)
        if index < 0:
            continue
        entry = hw.ROB[index]
        target = entry.PC + 1 + entry.dest_tag     # PC+4+offset<<2, in words
        update_btb(hw.btb, entry.PC, taken, target)
        correct_PC = target if taken else entry.PC + 1
        if correct_PC != entry.pred_PC:
            hw.stats['mispredicts'] += 1
            recover(hw, index)
            hw.PC.PC = correct_PC
            hw.PC.resume = cycle + 2
```

On a misprediction detected in cycle n, `recover()` carries out the three actions the handout lists: (1) it recovers the RAT from the architectural registers plus the surviving ROB entries; (2) it frees the reservation stations of wrong-path instructions and clears their wait-for tags, and removes them from the FUs, the load/store queue, the address adders, the memory unit, the results buffer and the CDB latches; (3) it clears the ROB entries after the branch. Fetch restarts at the correct PC in cycle $n+2$. For example, in `test1` at $W=1$ the first `Bne` resolves in cycle 9 and the next `Ld` issues in cycle 11. Wrong-path instructions never change architectural state: registers are written at commit, and stores only reach memory as the ROB head.

2.6 Loads and stores (`mem.py`)

I kept the original load/store queue logic: forwarding from the nearest older store, and loads going to memory only when every older store address is known and none matches. Two changes:

- A store is sent to memory (it commits) only when it is the head of the ROB. The original intended this with `check_if_sd_committable()`, but never actually called it (bug fix 2). Without the fix, speculative stores would corrupt memory.

Initial version

```
def check_if_sd_committable(ld_sd_queue, ROB):
    flag = False
    if ld_sd_queue[0].ldsd_tag == ROB[0].dest_tag:
        flag = True
    if ld_sd_queue[0].ldsd_tag == ROB[1].dest_tag:
        # previous ins has been broadcasted
        if len(ROB[0].cdb)!=0:
            flag = True
    return flag
```

My version

```
def check_if_sd_committable(ld_sd_queue, ROB):
    flag = False
    if (len(ROB) > 0) and (ld_sd_queue[0].ld_sd_tag == ROB[0].dest_tag):
        flag = True
    return flag
```

- Forwarding needs the load address from an earlier cycle, so that MEM follows EX. Addresses are range-checked, and a bad address on a wrong-path load is reported only if that load commits.

2.7 Input, output and the main loop (`init.py`, `main.py`, `print_status.py`)

- `read_input()` parses the input file: the hardware table, the parameters (`ROB entries`, `Issue width`, `CDB width` and `Commit width`), the initial registers and memory, and the program. `read_instruction()` normalises instructions (case-insensitive, commas optional), so the rest of the code still sees `"Ld F4 8(R1)"`.
- `main.py` builds the reservation stations, FU lists, CDBs and ROB from that configuration (values that were hard-coded before). It calls the stages in the original order (ISSUE, EX, MEM, CDB, COMMIT) plus `resolve_branches()` after EX. It also keeps running until the last store has written memory.
- The final table, register values and non-zero memory are printed in the required format. Command-line options `-w`, `-cdb` and `-commit` override the file.

2.8 Bugs fixed in the provided code

These all appeared once several instructions, or speculative ones, were in flight. Each fix is marked with a comment in the code.

1. `wb.py`: a broadcast always wrote its value into the RAT, even when a younger instruction had already renamed that register. With two writers of `F2` in flight (as in `test7`), later readers then got the older value.
2. `mem.py`: `if (check_if_sd_committable):` tests the function object, which is always true, instead of calling it. The function also read a non-existent field (`ldsd_tag`). So stores wrote memory before they committed.
3. `mem.py`: the load check used `ld_sd_queue[0].data` instead of the load's own entry, so a store with data at the queue head blocked every load.
4. `mem.py`: the forwarding loop overwrote its loop counter `index` with a ROB index, so later loads were compared against the wrong stores.
5. `exe.py`: fetching a "waiting" instruction called `fu_exe()` a second time in the same cycle, which advanced every instruction already in that unit by 2 cycles.
6. `exe.py`: `ld_sd_execution()` only advanced its counter in the first cycle, so an address calculation longer than 1 cycle never finished.
7. `issue.py`: the base register of `Ld/Sd` was taken from fixed string positions, which only works for `R1`–`R9`.
8. Branch offsets were treated as bytes (`offset/4`). The ISA defines `PC+4+offset«2`, so they are now in words. Branches also committed as soon as EX *started*, with no commit cycle.
9. R0 could be renamed and written. The simulation stopped while the last store was still writing memory.

3 Timing

The original timing conventions are kept: EX starts the cycle after the operands are broadcast; a result is broadcast the cycle after EX (or MEM) ends, through the CDB latch; an instruction commits the cycle after its broadcast; a load's MEM starts the cycle after its address is computed (1 cycle if forwarded). New rules:

- at most W issues and commit-width commits per cycle;
- at most one broadcast per CDB per cycle;
- a store's MEM interval starts in its commit cycle;
- a branch commits the cycle after its EX;
- a misprediction resolved at the end of EX in cycle n restarts issue in cycle $n+2$.

4 Verification

`tools/checks.py` checks every simulation in two ways:

1. **Results**: final registers, all 256 memory cells and the sequence of committed instructions must equal those of a sequential interpreter (`tools/reference.py`). This proves no wrong-path instruction committed and none was lost.
2. **Timing**:
   - issue and commit happen in program order;
   - per cycle, at most W issues, commit-width commits and CDB-width broadcasts;
   - ISSUE $<$ EX $<$ (MEM) $<$ WB $<$ COMMIT, with the configured EX and MEM lengths;
   - every instruction starts EX after its producers broadcast;
   - a store commits after its data is broadcast;
   - memory is used by one access at a time.

`tools/run_tests.py` runs the 9 test benches at $W = 1..4$, with one CDB and with W CDBs (72 runs). `tools/fuzz.py` generates random programs (ALU operations, loads and stores to overlapping addresses, forward branches, counted loops) on random machines (ROB 2–64, 1–4 RS, 1–2 FUs, random latencies and CDB count) and runs each at all four widths. Results:

- **Directed tests:** all 72 runs pass.
- **Random tests:** all 20,000 runs pass. They exercised about 52,000 mispredictions, 109,000 squashed instructions and 13,000 store-to-load forwards.
- **Bugs found by fuzzing:** the fuzzer found one bug in my first version of `recover()`. It cancelled a *committed* store still writing memory, because the store's old ROB tag had been reused by a squashed instruction.
- **Checking the checker:** I re-inserted bug 1 and two other deliberate bugs; each one is detected.

5 Test cases

Test	Purpose
`test0`	Configuration from the handout (`Issue width = 2`). The handout's program loops forever (R2≠R3), so a decrement of R2 was added; the loop runs twice.
`test1`	The loop provided with the original code (Ld/Mult/Ld/Add/Sd over 3 elements); branch offset −8 in words.
`test2`	16 independent integer and FP instructions on two units of each type: best case for multi-issue.
`test3`	A chain of true (RAW) dependences plus WAR/WAW reuse of registers.
`test4`	Store-to-load forwarding, a load waiting for an unknown store address, two stores to one address.
`test5`	Nested loops with `Beq`/`Bne`; wrong-path code that would corrupt R9/F9/memory if squashing failed.
`test6`	Structural hazards: 4-entry ROB, 1–2 RS per unit, 2-cycle unpipelined integer adder.
`test7`, `test8`	"Test Case #01/#02" of the sample report, for comparison.

6 Results under different issue widths

All runs end with the correct registers and memory (Section 6.7). Full outputs for every test, width and CDB setting are in `results/`.

		CDB width = 1				CDB width = W			Best
Test	Instr.	W=1	W=2	W=3	W=4	W=2	W=3	W=4	speedup
`test0`	8	18	15	13	13	15	13	13	1.38×
`test1`	25	55	53	52	52	53	52	52	1.06×
`test2`	16	24	21	20	19	15	13	10	2.40×
`test3`	10	31	28	27	27	28	27	26	1.19×
`test4`	13	27	25	25	24	24	24	23	1.17×
`test5`	37	66	57	58	56	54	54	54	1.22×
`test6`	10	28	28	28	28	28	28	28	1.00×
`test7`	17	36	35	35	35	33	33	33	1.09×
`test8`	17	49	47	47	47	47	47	47	1.04×

Table 1: Total cycles (cycle of the last commit). Best speedup = cycles at W=1 / fewest cycles over all settings.

	CDB width = 1				CDB width = W			mispred./squashed	
Test	W=1	W=2	W=3	W=4	W=2	W=3	W=4	W=1	W=4
`test0`	0.44	0.53	0.62	0.62	0.53	0.62	0.62	2/2	2/3
`test1`	0.45	0.47	0.48	0.48	0.47	0.48	0.48	2/2	2/1
`test2`	0.67	0.76	0.80	0.84	1.07	1.23	1.60	0/0	0/0
`test3`	0.32	0.36	0.37	0.37	0.36	0.37	0.38	0/0	0/0
`test4`	0.48	0.52	0.52	0.54	0.54	0.54	0.57	0/0	0/0
`test5`	0.56	0.65	0.64	0.66	0.69	0.69	0.69	8/17	8/39
`test6`	0.36	0.36	0.36	0.36	0.36	0.36	0.36	0/0	0/0
`test7`	0.47	0.49	0.49	0.49	0.52	0.52	0.52	0/0	0/0
`test8`	0.35	0.36	0.36	0.36	0.36	0.36	0.36	0/0	0/0

Table 2: IPC, and branch mispredictions / squashed instructions at W=1 and W=4 (1 CDB).

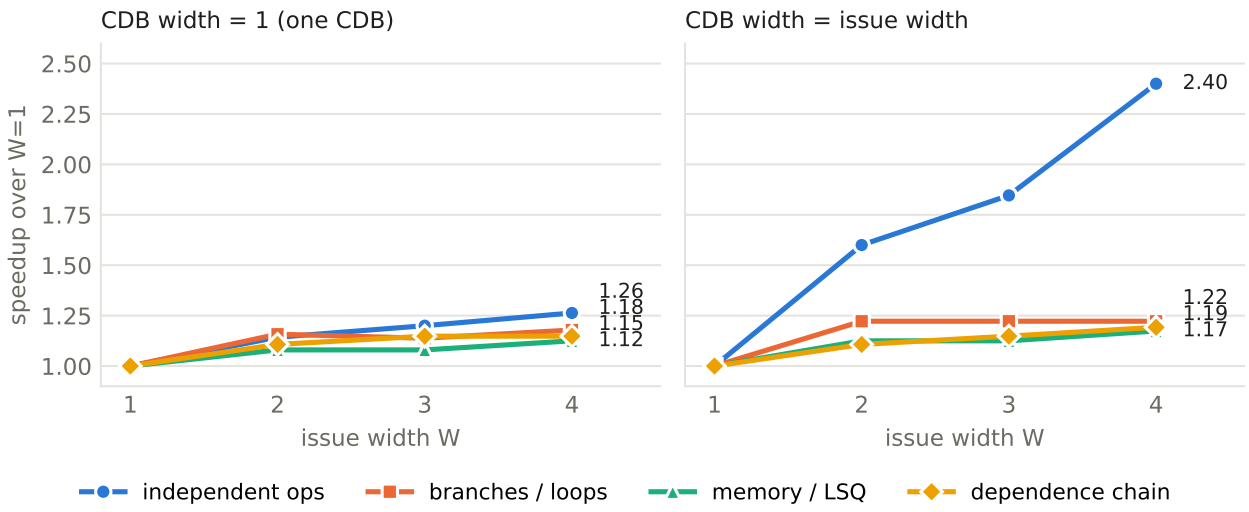


Figure 1: Speedup over single issue for four representative tests.

Discussion.

- **Independent instructions** (`test2`): 24 → 10 cycles at W=4 with 4 CDBs (IPC 0.67 → 1.60). With the single CDB of the handout, the same machine needs 19 cycles: with one result per cycle, the CDB becomes the bottleneck.
- **Dependence chains and long latencies** (`test1`, `test3`, `test7`, `test8`): little gain. Instructions issue earlier but still wait for their operands, often 15–20 cycle multiplies.
- **Branches** (`test0`, `test5`): more instructions issue between mispredictions. At W=3 with one CDB, `test5` takes 58 cycles against 57 at W=2. The wider front end issues more wrong-path instructions (42 squashed vs. 31), and these compete for functional units and the CDB before they are squashed.
- **Structural limits** (`test6`): the 4-entry ROB caps the window, so every width takes 28 cycles.
- **Sample report cases**: at W=1, `test7` takes 36 cycles (sample report: 36). At W=2 with 2 CDBs it takes 33 (sample: 33). `test8` takes 49/47 (sample: 47/46). Individual cycles differ because of the bug fixes above (for example, the FP units no longer advance twice in one cycle).

6.1 Test 0: handout configuration, $W = 1..4$

At W=4 the whole loop body issues in cycle 1. The first `Bne` (BTB miss, predicted not taken, actually taken) resolves in cycle 4, and the loop restarts in cycle 6. The second `Bne` is predicted taken by the BTB, but the loop exits, so it also mispredicts.

W = 1

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4	–	6	7
`Ld F4 8(R1)`	2	3–3	4–7	8	9
`Addi R2 R2 -10`	3	4–4	–	5	10
`Bne R2 R3 -4`	4	6–6	–	–	11
`Add.d F1 F2 F3`	8	9–11	–	13	14
`Ld F4 8(R1)`	9	10–10	11–14	15	16
`Addi R2 R2 -10`	10	11–11	–	12	17
`Bne R2 R3 -4`	11	13–13	–	–	18

W = 2

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4	–	5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	2	3–3	–	4	8
`Bne R2 R3 -4`	2	5–5	–	–	9
`Add.d F1 F2 F3`	7	8–10	–	11	12
`Ld F4 8(R1)`	7	8–8	9–12	13	14
`Addi R2 R2 -10`	8	9–9	–	10	14
`Bne R2 R3 -4`	8	11–11	–	–	15

W = 3

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4	–	5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	1	2–2	–	3	8
`Bne R2 R3 -4`	2	4–4	–	–	8
`Add.d F1 F2 F3`	6	7–9	–	10	11
`Ld F4 8(R1)`	6	7–7	8–11	12	13
`Addi R2 R2 -10`	6	7–7	–	8	13
`Bne R2 R3 -4`	7	9–9	–	–	13

W = 4

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4	–	5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	1	2–2	–	3	8
`Bne R2 R3 -4`	1	4–4	–	–	8
`Add.d F1 F2 F3`	6	7–9	–	10	11
`Ld F4 8(R1)`	6	7–7	8–11	12	13
`Addi R2 R2 -10`	6	7–7	–	8	13
`Bne R2 R3 -4`	6	9–9	–	–	13

Table 3: `test0` (1 CDB). EX/MEM give first–last cycle; "–" = stage not used.

6.2 Test 2: independent instructions

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Add R10 R1 R2`	1	4	1	4	1	4	1	4
`Add.d F10 F1 F2`	2	7	1	5	1	5	1	5
`Sub R11 R3 R4`	3	8	2	5	1	5	1	5
`Mult.d F11 F3 F4`	4	12	2	9	2	9	1	7
`Addi R12 R5 100`	5	13	3	9	2	9	2	7
`Sub.d F12 F5 F6`	6	14	3	10	2	9	2	7
`Add R13 R6 R1`	7	15	4	10	3	10	2	7
`Add.d F13 F2 F3`	8	16	4	11	3	10	2	8
`Sub R14 R2 R5`	9	17	5	11	3	10	3	8
`Mult.d F14 F5 F6`	10	18	5	12	4	11	3	9
`Addi R15 R3 -7`	11	19	6	12	4	11	3	9
`Sub.d F15 F1 F4`	12	20	6	13	4	11	3	9
`Add R16 R4 R6`	13	21	7	13	5	12	4	9
`Add.d F16 F4 F5`	14	22	7	14	5	12	4	10
`Addi R17 R1 42`	15	23	8	14	5	12	4	10
`Mult.d F17 F1 F6`	16	24	8	15	6	13	4	10
Total	24 cycles		15 cycles		13 cycles		10 cycles	

Table 4: `test2`: ISSUE (I) and COMMIT (C) cycles, CDB width $= W$. Exactly W instructions issue per cycle.

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Add R10 R1 R2`	1	4	1	4	1	4	1	4
`Add.d F10 F1 F2`	2	7	1	6	1	7	1	8
`Sub R11 R3 R4`	3	8	2	6	1	7	1	8
`Mult.d F11 F3 F4`	4	12	2	12	2	16	1	16
`Addi R12 R5 100`	5	13	3	12	2	16	2	16
`Sub.d F12 F5 F6`	6	14	3	13	2	16	2	16
`Add R13 R6 R1`	7	15	4	13	3	17	2	16
`Add.d F13 F2 F3`	8	16	4	14	3	17	2	17
`Sub R14 R2 R5`	9	17	5	14	3	17	3	17
`Mult.d F14 F5 F6`	10	18	5	18	4	18	3	18
`Addi R15 R3 -7`	11	19	6	18	4	18	3	18
`Sub.d F15 F1 F4`	12	20	6	19	4	18	3	18
`Add R16 R4 R6`	13	21	7	19	5	19	4	18
`Add.d F16 F4 F5`	14	22	7	20	5	19	4	19
`Addi R17 R1 42`	15	23	8	20	5	19	4	19
`Mult.d F17 F1 F6`	16	24	8	21	6	20	4	19
Total	24 cycles		21 cycles		20 cycles		19 cycles	

Table 5: `test2` with one CDB: issue still reaches W per cycle, but write-back is serialized.

6.3 Test 1: loop with speculation, $W = 1$ vs. $W = 4$

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	9	10
`Mult.d F4 F2 F20`	2	10–24	–	25	26
`Ld F6 0(R2)`	3	4–4	8–12	13	27
`Add.d F6 F4 F6`	4	26–29	–	30	31
`Sd F6 0(R2)`	5	6–6	33–37	–	33
`Addi R1 R1 -4`	6	7–7	–	8	34
`Addi R2 R2 -4`	7	8–8	–	10	35
`Bne R1 R0 -8`	8	9–9	–	–	36
`Ld F2 0(R1)`	11	12–12	13–17	19	37
`Mult.d F4 F2 F20`	12	20–34	–	35	38
`Ld F6 0(R2)`	13	14–14	18–22	23	39
`Add.d F6 F4 F6`	14	36–39	–	40	41
`Sd F6 0(R2)`	15	16–16	42–46	–	42
`Addi R1 R1 -4`	16	17–17	–	18	43
`Addi R2 R2 -4`	17	18–18	–	20	44
`Bne R1 R0 -8`	18	19–19	–	–	45
`Ld F2 0(R1)`	19	20–20	23–27	28	46
`Mult.d F4 F2 F20`	20	29–43	–	44	47
`Ld F6 0(R2)`	21	22–22	28–32	33	48
`Add.d F6 F4 F6`	22	45–48	–	49	50
`Sd F6 0(R2)`	23	24–24	51–55	–	51
`Addi R1 R1 -4`	24	25–25	–	26	52
`Addi R2 R2 -4`	25	26–26	–	27	53
`Bne R1 R0 -8`	26	27–27	–	–	54
`Add.d F20 F2 F2`	29	30–33	–	34	55

Table 6: `test1`, W=1 (1 CDB).

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23	–	24	25
`Ld F6 0(R2)`	1	3–3	8–12	13	25
`Add.d F6 F4 F6`	1	25–28	–	29	30
`Sd F6 0(R2)`	2	4–4	33–37	–	33
`Addi R1 R1 -4`	2	3–3	–	4	33
`Addi R2 R2 -4`	2	4–4	–	5	33
`Bne R1 R0 -8`	2	5–5	–	–	33
`Ld F2 0(R1)`	7	8–8	13–17	18	34
`Mult.d F4 F2 F20`	7	19–33	–	35	36
`Ld F6 0(R2)`	7	9–9	18–22	23	36
`Add.d F6 F4 F6`	7	36–39	–	40	41
`Sd F6 0(R2)`	8	10–10	42–46	–	42
`Addi R1 R1 -4`	8	9–9	–	10	42
`Addi R2 R2 -4`	8	10–10	–	11	42
`Bne R1 R0 -8`	8	11–11	–	–	42
`Ld F2 0(R1)`	9	11–11	23–27	28	43
`Mult.d F4 F2 F20`	10	29–43	–	44	45
`Ld F6 0(R2)`	13	14–14	28–32	34	45
`Add.d F6 F4 F6`	13	45–48	–	49	50
`Sd F6 0(R2)`	18	19–19	51–55	–	51
`Addi R1 R1 -4`	18	19–19	–	20	51
`Addi R2 R2 -4`	18	20–20	–	21	51
`Bne R1 R0 -8`	18	21–21	–	–	51
`Add.d F20 F2 F2`	26	29–32	–	33	52

Table 7: `test1`, W=4 (1 CDB). The long multiplies dominate: 55 $\rightarrow$ 52 cycles.

6.4 Test 4: forwarding and memory disambiguation, $W = 1$ and $W = 2$

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7	–	8	9
`Sd F3 0(R1)`	2	3–3	10–13	–	10
`Ld F4 0(R1)`	3	4–4	9	10	11
`Ld F5 8(R1)`	4	5–5	6–9	11	12
`Add.d F6 F4 F5`	5	12–13	–	14	15
`Sd F6 8(R2)`	6	7–7	16–19	–	16
`Sd F1 8(R2)`	7	8–8	20–23	–	20
`Ld F7 24(R0)`	8	9–9	10	13	21
`Add R4 R1 R2`	9	10–10	–	12	22
`Sd F7 16(R4)`	10	13–13	24–27	–	24
`Ld F8 40(R0)`	11	12–12	14	15	25
`Ld F9 0(R3)`	12	14–14	15	16	26
`Add.d F10 F8 F9`	13	17–18	–	19	27

Table 8: `test4`, W=1. One-cycle MEM entries are store-to-load forwards.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7	–	8	9
`Sd F3 0(R1)`	1	2–2	10–13	–	10
`Ld F4 0(R1)`	2	3–3	9	11	12
`Ld F5 8(R1)`	2	4–4	5–8	10	12
`Add.d F6 F4 F5`	3	12–13	–	14	15
`Sd F6 8(R2)`	3	5–5	16–19	–	16
`Sd F1 8(R2)`	4	6–6	20–23	–	20
`Ld F7 24(R0)`	4	7–7	8	9	20
`Add R4 R1 R2`	5	6–6	–	7	21
`Sd F7 16(R4)`	5	8–8	24–27	–	24
`Ld F8 40(R0)`	9	10–10	11	12	24
`Ld F9 0(R3)`	10	11–11	12	13	25
`Add.d F10 F8 F9`	10	14–15	–	16	25

Table 9: `test4`, W=2.

6.5 Tests 3, 6, 7 and 8: issue/commit cycles per width

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Addi R2 R1 1`	1	4	1	4	1	4	1	4
`Add R3 R2 R2`	2	6	1	6	1	6	1	6
`Sub R4 R3 R1`	3	8	2	8	1	8	1	8
`Add.d F3 F1 F2`	4	9	2	8	2	8	1	8
`Mult.d F4 F3 F3`	5	15	3	13	2	13	2	12
`Sub.d F5 F4 F1`	6	19	3	17	2	17	2	16
`Add.d F3 F5 F5`	7	23	4	21	3	21	2	20
`Mult.d F1 F3 F2`	8	29	4	27	3	27	2	26
`Addi R2 R4 10`	9	30	5	27	3	27	3	26
`Add R4 R2 R2`	10	31	5	28	4	27	3	26
Total	31 cycles		28 cycles		27 cycles		26 cycles	

Table 10: `test3` (dependence chain, CDB width = W).

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Ld F3 0(R1)`	1	7	1	7	1	7	1	7
`Ld F4 4(R1)`	2	10	1	10	1	10	1	10
`Add.d F5 F3 F4`	3	14	2	14	1	14	1	14
`Mult.d F6 F5 F2`	4	21	2	21	2	21	1	21
`Add.d F7 F1 F2`	11	22	11	21	11	21	11	21
`Mult.d F8 F1 F1`	15	23	15	23	15	23	15	23
`Addi R2 R1 4`	16	24	15	23	15	23	15	23
`Addi R3 R2 4`	21	25	21	25	21	25	21	25
`Sd F6 0(R3)`	22	26	21	26	21	26	21	26
`Sub R4 R3 R1`	23	28	23	28	23	28	23	28
Total	28 cycles		28 cycles		28 cycles		28 cycles	

Table 11: `test6` (4-entry ROB, 1 CDB): issue stalls in order when the ROB or a reservation station is full.

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Ld F1 0(R1)`	1	10	1	10	1	10	1	10
`Mult.d F2 F1 F0`	2	16	1	16	1	16	1	16
`Add.d F3 F3 F2`	3	20	2	20	1	20	1	20
`Mult.d F2 F1 F1`	4	21	2	20	2	20	1	20
`Add.d F4 F2 F4`	5	22	3	21	2	21	2	21
`Addi R1 R1 8`	6	23	3	21	2	21	2	21
`Ld F1 0(R1)`	7	24	4	22	3	21	2	21
`Mult.d F2 F1 F0`	8	25	4	22	3	22	2	22
`Add.d F3 F3 F2`	9	28	5	26	3	26	3	26
`Mult.d F2 F1 F1`	10	29	5	26	4	26	3	26
`Add.d F4 F2 F4`	11	30	6	27	4	27	3	27
`Addi R1 R1 8`	12	31	6	27	4	27	3	27
`Ld F1 0(R1)`	13	32	7	28	5	27	4	27
`Mult.d F2 F1 F0`	14	33	11	28	11	28	11	28
`Add.d F3 F3 F2`	17	34	17	32	17	32	17	32
`Mult.d F2 F1 F1`	18	35	17	32	17	32	17	32
`Add.d F4 F2 F4`	19	36	18	33	18	33	18	33
Total	36 cycles		33 cycles		33 cycles		33 cycles	

Table 12: `test7` (sample report Test Case #01, CDB width = W).

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Ld F1 0(R1)`	1	10	1	9	1	9	1	9
`Mult.d F2 F1 F0`	2	26	1	25	1	25	1	25
`Add.d F3 F3 F2`	3	30	2	29	1	29	1	29
`Mult.d F2 F1 F1`	4	31	2	29	2	29	1	29
`Ld F1 0(R1)`	5	32	3	30	2	29	2	29
`Addi R1 R1 4`	6	33	3	30	2	30	2	29
`Add.d F4 F2 F4`	7	34	4	31	3	30	2	30
`Addi R1 R1 8`	8	35	4	31	3	30	2	30
`Ld F1 0(R1)`	9	36	5	32	3	31	3	30
`Mult.d F2 F1 F0`	11	37	10	35	10	35	10	35
`Add.d F3 F3 F2`	12	39	10	39	10	39	10	39
`Mult.d F2 F1 F1`	13	40	11	39	10	39	10	39
`Ld F1 0(R1)`	14	41	11	40	11	39	10	39
`Mult.d F2 F1 F0`	20	42	20	40	20	40	20	40
`Add.d F3 F3 F2`	27	44	26	44	26	44	26	44
`Mult.d F2 F1 F1`	28	45	26	44	26	44	26	44
`Add.d F4 F2 F4`	29	49	27	47	26	47	26	47
Total	49 cycles		47 cycles		47 cycles		47 cycles	

Table 13: `test8` (sample report Test Case #02, CDB width = W).

6.6 Test 5: branches at $W = 2$

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Addi R2 R0 2`	1	2–2	–	3	4
`Add.d F3 F3 F1`	1	2–3	–	4	5
`Addi R2 R2 -1`	2	4–4	–	5	6
`Bne R2 R0 -3`	2	6–6	–	–	7
`Add.d F3 F3 F1`	8	9–10	–	11	12
`Addi R2 R2 -1`	8	9–9	–	10	12
`Bne R2 R0 -3`	9	11–11	–	–	13
`Mult.d F4 F3 F2`	13	14–17	–	18	19
`Addi R1 R1 -1`	13	14–14	–	15	19
`Beq R1 R0 1`	14	16–16	–	–	20
`Beq R0 R0 -8`	14	15–15	–	–	20
`Addi R2 R0 2`	17	18–18	–	19	21
`Add.d F3 F3 F1`	17	18–19	–	20	21
`Addi R2 R2 -1`	18	20–20	–	21	22
`Bne R2 R0 -3`	18	22–22	–	–	23
`Add.d F3 F3 F1`	24	25–26	–	27	28
`Addi R2 R2 -1`	24	25–25	–	26	28
`Bne R2 R0 -3`	25	27–27	–	–	29
`Mult.d F4 F3 F2`	29	30–33	–	36	37
`Addi R1 R1 -1`	29	30–30	–	31	37
`Beq R1 R0 1`	30	32–32	–	–	38
`Beq R0 R0 -8`	30	31–31	–	–	38
`Addi R2 R0 2`	31	33–33	–	34	39
`Add.d F3 F3 F1`	31	32–33	–	35	39
`Addi R2 R2 -1`	32	35–35	–	38	40
`Bne R2 R0 -3`	32	39–39	–	–	40
`Add.d F3 F3 F1`	41	42–43	–	44	45
`Addi R2 R2 -1`	41	42–42	–	43	45
`Bne R2 R0 -3`	42	44–44	–	–	46
`Mult.d F4 F3 F2`	46	47–50	–	52	53
`Addi R1 R1 -1`	46	47–47	–	48	53
`Beq R1 R0 1`	47	49–49	–	–	54
`Addi R9 R0 7`	48	50–50	–	51	54
`Beq R9 R0 2`	48	52–52	–	–	55
`Add.d F9 F3 F4`	49	53–54	–	55	56
`Sd F9 200(R0)`	49	50–50	57–58	–	57
`Addi R10 R9 1`	50	53–53	–	54	57

Table 14: `test5`, W=2 (1 CDB), committed instructions; 31 wrong-path instructions were squashed. Each gap in ISSUE after a branch is the $n \to n{+}2$ restart.

6.7 Final register and memory values

These values are identical for every width and CDB setting, and equal to the sequential interpreter's (non-zero values only):

- `test0_handout_example`. Registers: `R1=10`, `F1=30.1`, `F2=30.1`. Memory: `Mem[4]=1`, `Mem[8]=2`, `Mem[12]=3.4`.
- `test1_loop`. Registers: `R2=20`, `F2=3.0`, `F4=9.0`, `F6=15.0`, `F20=6.0`. Memory: `Mem[4]=3.0`, `Mem[8]=2.0`, `Mem[12]=1.0`, `Mem[24]=15.0`, `Mem[28]=11.0`, `Mem[32]=7.0`.
- `test2_independent`. Registers: `R1=1`, `R2=2`, `R3=3`, `R4=4`, `R5=5`, `R6=6`, `R10=3`, `R11=-1`, `R12=105`, `R13=7`, `R14=-3`, `R15=-4`, `R16=10`, `R17=43`, `F1=1.5`, `F2=2.5`, `F3=3.5`, `F4=4.5`, `F5=5.5`, `F6=6.5`, `F10=4.0`, `F11=15.75`, `F12=-1.0`, `F13=6.0`, `F14=35.75`, `F15=-3.0`, `F16=10.0`, `F17=9.75`. Memory: none.
- `test3_dependency_chain`. Registers: `R1=5`, `R2=17`, `R3=12`, `R4=34`, `F1=138.0`, `F2=3.0`, `F3=46.0`, `F4=25.0`, `F5=23.0`. Memory: none.
- `test4_memory`. Registers: `R1=8`, `R2=16`, `R3=40`, `R4=24`, `F1=1.5`, `F2=2.0`, `F3=3.0`, `F4=3.0`,

`F5=20.0,F6=23.0,F7=1.5,F8=1.5,F9=1.5,F10=3.0`. Memory: `Mem[8]=3.0,Mem[16]=20.0, Mem[24]=1.5,Mem[40]=1.5`.

- `test5_branches`. Registers: `R9=7,R10=8,F1=1.0,F2=0.5,F3=6.0,F4=3.0,F9=9.0`. Memory: `Mem[200]=9.0`.
- `test6_structural`. Registers: `R1=4,R2=8,R3=12,R4=8,F1=1.0,F2=2.0,F3=7.0,F4=8.0, F5=15.0,F6=30.0,F7=3.0,F8=1.0`. Memory: `Mem[4]=7.0,Mem[8]=8.0,Mem[12]=30.0`.
- `test7_sample_report_tc1`. Registers: `R1=28,R2=32,F1=5.0,F2=25.0,F4=26.0,F20=3.0`. Memory: `Mem[4]=3.0,Mem[8]=2.0,Mem[12]=1.0,Mem[24]=6.0,Mem[28]=5.0,Mem[32]=4.0`.
- `test8_sample_report_tc2`. Registers: `R1=24,R2=32,F1=6.0,F2=36.0,F4=37.0,F20=3.0`. Memory: `Mem[4]=3.0,Mem[8]=2.0,Mem[12]=1.0,Mem[24]=6.0,Mem[28]=5.0,Mem[32]=4.0`.

6.8 Example of the complete output

`python3 main.py ../tests/test1_loop.txt -w 2`:

```
Issue width = 2, commit width = 2, CDB width = 1, ROB entries = 64

                            ISSUE          EXE            MEM            WB             COMMIT
Ld F2 0(R1)                 [1]            [2, 2]         [3, 7]         [8]            [9]
Mult.d F4 F2 F20            [1]            [9, 23]        []             [24]           [25]
Ld F6 0(R2)                 [2]            [3, 3]         [8, 12]        [14]           [25]
Add.d F6 F4 F6              [2]            [25, 28]       []             [29]           [30]
Sd F6 0(R2)                 [3]            [4, 4]         [33, 37]       []             [33]
Addi R1 R1 -4               [3]            [4, 4]         []             [5]            [33]
Addi R2 R2 -4               [4]            [5, 5]         []             [6]            [34]
Bne R1 R0 -8                [4]            [6, 6]         []             []             [34]
Ld F2 0(R1)                 [8]            [9, 9]         [13, 17]       [18]           [35]
Mult.d F4 F2 F20            [8]            [19, 33]       []             [35]           [36]
Ld F6 0(R2)                 [9]            [10, 10]       [18, 22]       [23]           [36]
Add.d F6 F4 F6              [9]            [36, 39]       []             [40]           [41]
Sd F6 0(R2)                 [10]           [11, 11]       [42, 46]       []             [42]
Addi R1 R1 -4               [10]           [11, 11]       []             [12]           [42]
Addi R2 R2 -4               [11]           [12, 12]       []             [13]           [43]
Bne R1 R0 -8                [11]           [13, 13]       []             []             [43]
Ld F2 0(R1)                 [12]           [13, 13]       [23, 27]       [28]           [44]
Mult.d F4 F2 F20            [12]           [29, 43]       []             [44]           [45]
Ld F6 0(R2)                 [13]           [14, 14]       [28, 32]       [34]           [45]
Add.d F6 F4 F6              [13]           [45, 48]       []             [49]           [50]
Sd F6 0(R2)                 [18]           [19, 19]       [51, 55]       []             [51]
Addi R1 R1 -4               [18]           [19, 19]       []             [20]           [51]
Addi R2 R2 -4               [19]           [20, 20]       []             [21]           [52]
Bne R1 R0 -8                [19]           [21, 21]       []             []             [52]
Add.d F20 F2 F2             [26]           [29, 32]       []             [33]           [53]

REGISTER VALUES
        R0      R1      R2      R3      R4      R5      R6      R7
value:  0       0       20      0       0       0       0       0
        R8      R9      R10     R11     R12     R13     R14     R15
value:  0       0       0       0       0       0       0       0
        R16     R17     R18     R19     R20     R21     R22     R23
value:  0       0       0       0       0       0       0       0
        R24     R25     R26     R27     R28     R29     R30     R31
value:  0       0       0       0       0       0       0       0
        F0      F1      F2      F3      F4      F5      F6      F7
value:  0.0     0.0     3.0     0.0     9.0     0.0     15.0    0.0
        F8      F9      F10     F11     F12     F13     F14     F15
value:  0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
        F16     F17     F18     F19     F20     F21     F22     F23
value:  0.0     0.0     0.0     0.0     6.0     0.0     0.0     0.0
        F24     F25     F26     F27     F28     F29     F30     F31
value:  0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0

NON-ZERO MEMORY VALUES
Addresses      Values
```

```
4               3.0
8               2.0
12              1.0
24              15.0
28              11.0
32              7.0

Total cycles: 53, instructions committed: 25, IPC: 0.47, branch mispredictions: 2, squashed
    instructions: 1
```

7 How to run

The code, test cases and results are in the GitHub repository `https://github.com/CoinismRS/LipskyIlan_COSC6385_Programming_Tomasulo_HW`.

```
git clone https://github.com/CoinismRS/LipskyIlan_COSC6385_Programming_Tomasulo_HW.git
cd LipskyIlan_COSC6385_Programming_Tomasulo_HW
cd code
python3 main.py ../tests/test1_loop.txt          # issue width from the file
python3 main.py ../tests/test1_loop.txt -w 4     # override issue width (1-4)
python3 main.py ../tests/test2_independent.txt -w 4 --cdb 4
python3 ../tools/run_tests.py                    # all tests, W=1..4, checked
python3 ../tools/fuzz.py 2000                    # random programs vs. reference
```

Python 3 only, no other packages (the report build also needs pdflatex and matplotlib). See `README.md` for the input format.